파이썬 완전 백과사전 2권 · 딥러닝 심화: 언어와 생성

5일차

고르는 모델에서 그리는 모델로

21~25차시 · 책 15장 생성 모델: 오토인코더, VAE, 확산 모델

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/15-generative-models · 과정 마지막 날

오늘의 시간표

차시	주제	책 범위	오래 걸리는 예제
21	오토인코더: 줄였다가 되살리기, 잠재 공간	15장 들어가며 ~ 15.2	b01 27초 · b02 13초
22	잡음 제거, 이상 탐지, 오토인코더로 그리기	15.3 ~ 15.5	b05 잡음 제거 73초
23	VAE: 점 대신 구름으로, GAN의 생각	15.6 ~ 15.7	b09 VAE 8초
24	확산 모델 만들기, 생성 모델 평가	15.8 ~ 15.9	b13 학습 110초 · b14 그리기 45초
25	실습: 주문한 옷을 그리는 조건부 확산 · 과정 마무리	15.10 ~ 15.11	b16 학습 123초 · b17 채점 47초

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분 · 오래 걸리는 학습은 차시 첫머리에 실행을 걸어 둔다

준비: 15-generative-models 폴더로 이동 · 생성도구.py, 확산도구.py가 같은 폴더에 있어야 한다 · FashionMNIST는 첫 실행 때 data/로 내려받는다(약 82MB)

21차시

21

오토인코더: 줄였다가 되살리기

책 15장 들어가며 ~ 15.2절

학습 목표

- 고르는 모델(판별)과 그리는 모델(생성)의 차이를 말한다
- 인코더, 잠재 벡터, 디코더로 오토인코더를 설명하고 직접 학습시킨다
- 잠재 크기를 바꾸며 PCA와 비교하고, 2차원 잠재 공간을 그림으로 읽는다

15장 고르는 모델, 그리는 모델

감상자: 판별 모델

사진을 보고 "이건 앵클부츠"라고 열 가지 중 하나를 **고른다**

그림 사이의 **경계**만 알면 된다

화가: 생성 모델

"앵클부츠를 그려 줘"에 세상에 없던 사진을 **그린다**

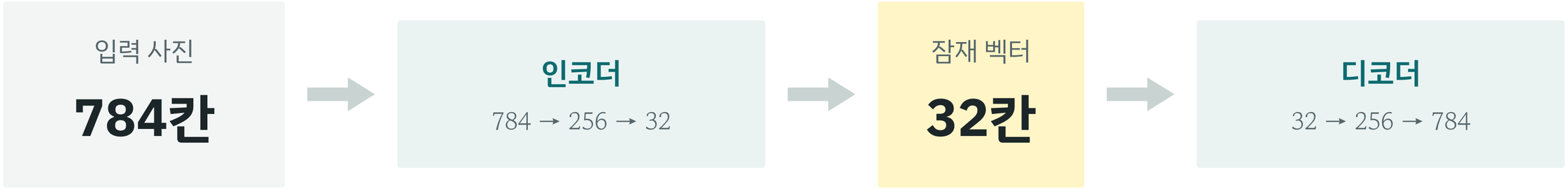
사진이 **어떻게 생겼는지(분포)** 자체를 배워야 한다

1 오토인코더 · 21~22차시
줄였다가 되살리기

2 VAE · 23차시
처음으로 새 그림

3 확산 모델 · 24~25차시
잡음을 걷어 내며 그리기

15.1 요약 노트로 원본 다시 쓰기



정답 = 입력 자기 자신. 레이블이 필요 없는 비지도 학습

가운데가 좁은 **병목**이라 무엇이 중요한지 골라 담는 법을 배운다

15.1 이 장의 도구 모음: 생성도구.py

이름	하는 일	처음 쓰는 곳
사진_불러오기	FashionMNIST를 통째로 (N, 1, 28, 28), 값 0~1 텐서로	15.1
로더_만들기	사진만 담은 DataLoader (정답이 필요 없다)	15.1, 15.6
격자_저장	사진 묶음을 바둑판 PNG 한 장으로, 3배 키워서	모든 그림
오토인코더	784 → 256 → 잠재_크기 → 256 → 784, 끝에 Sigmoid	15.1
오토인코더_학습	손실 = MSE(모델(입력), x). 잡음 > 0이면 잡음 섞은 입력	15.1, 15.3
채점기_만들기	그린 그림을 채점할 작은 CNN 분류기 (2에폭)	15.9

모든 예제가 **import**해서 쓴다: 예제 파일들을 이 파일과 **같은 폴더**에 둔다

15.1 첫 오토인코더: 784칸을 32칸으로

```
모델 = 오토인코더(잠재_크기=32)
오토인코더_학습(모델, X_학습, 에폭=5)

모델.eval()
with torch.no_grad():
    잠재 = 모델.인코더(X_시험[:8])      # 사진 8장을 32칸으로 줄이고
    복원 = 모델.디코더(잠재)            # 다시 784칸으로 되살린다
    시험_손실 = F.mse_loss(모델(X_시험), X_시험).item()
```

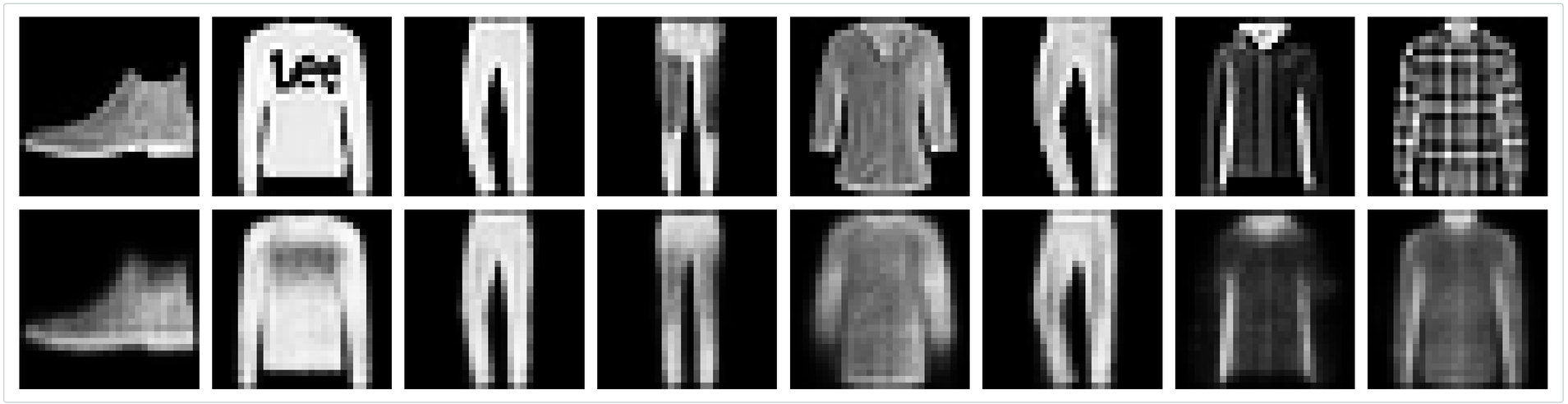
```
파라미터 419,120개
에폭 1: 재구성 손실 0.0405
에폭 5: 재구성 손실 0.0137
잠재 벡터 모양: (8, 32)
시험 사진 재구성 손실: 0.0134
```

학습 0.0137, 시험 **0.0134**: 외운 게 아니라 요약하는 법을 배웠다

$\sqrt{0.0134} \approx 0.12$: 한 칸당 밝기가 평균 0.12쯤 어긋난다

21차시 · 15.1 첫 오토인코더 · 예제 15-generative-models/b01_ae_train.py (핵심 부분, 출력 일부)

15.1 되살린 사진: 윤곽은 살고 잔무늬는 뭉개진다

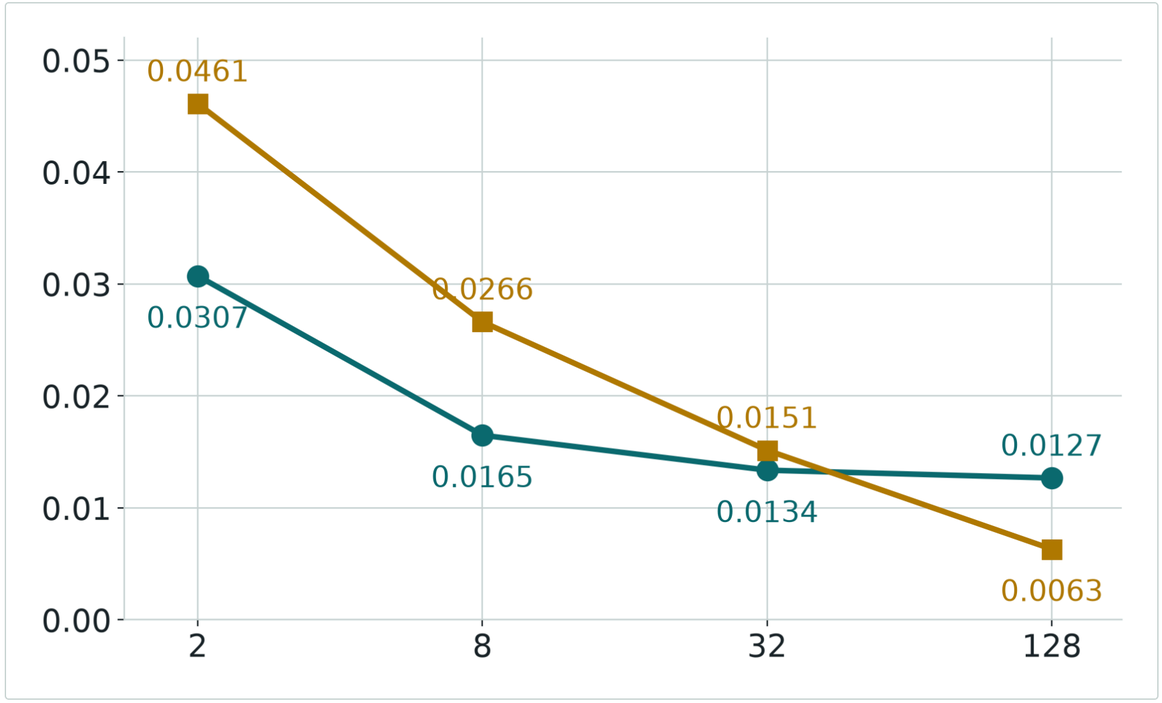


신발, 바지, 셔츠의 **전체 모양**은 거의 그대로

티셔츠의 로고 글자, 셔츠의 체크무늬 같은 **잔무늬**는 흐릿하게

15.1 얼마나 줄일 수 있을까: PCA와 겨루기

● 오토인코더 · ■ PCA · 세로는 시험 재구성 손실(MSE)



가로는 잠재 크기 k (2, 8, 32, 128)

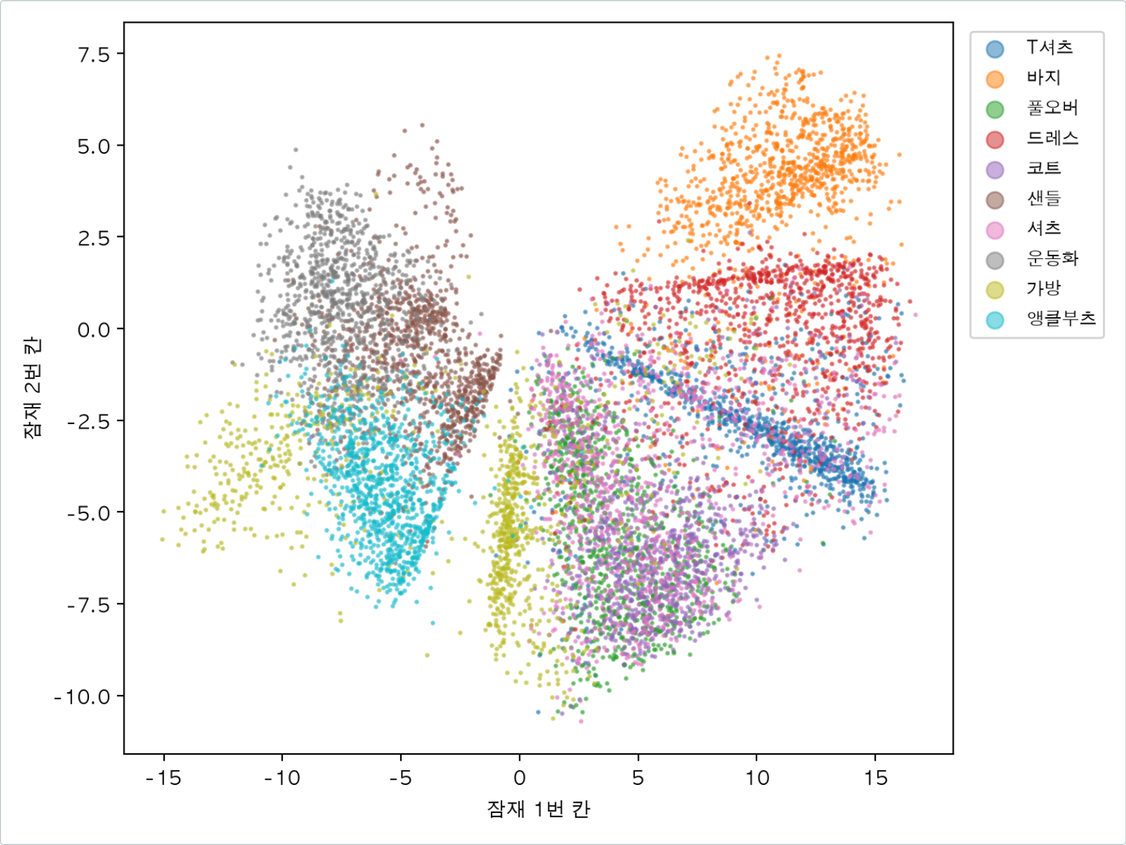
아주 작게 줄이면 오토인코더가 유리: 2칸, 8칸에서 PCA보다 30% 넘게 낮다. ReLU가 구부러진 모양을 접는다

넉넉하게 줄이면 PCA가 이긴다: 128칸 0.0063 대 0.0127

순서: **단순한 방법(PCA)부터**, 부족하면 오토인코더

21차시 · 15.1 얼마나 줄일 수 있을까 · 책의 b02_latent_size.py를 같은 설정으로 다시 돌려 그린 그림 (숫자는 출력과 같다)

15.2 잠재 공간 들여다보기: 사진 한 장 = 점 하나



종류	중심 (가로, 세로)
운동화	(-7.5, 0.4)
바지	(10.8, 3.6)
풀오버	(4.2, -5.6)
코트	(6.0, -5.5)
셔츠	(5.7, -4.1)

정답을 **한 번도 안 봤는데** 신발은 왼쪽, 옷은 오른쪽

풀오버·코트·셔츠는 2칸 남짓 거리로 뒤섞였다

21차시 · 15.2 잠재 공간 들여다보기 · 예제 b03_latent_2d.py가 만든 latent_2d.png와 출력(일부)

직접 실행 10분

1. b01_ae_train.py와 b03_latent_2d.py를 실행하고, ae_recon.png와 latent_2d.png를 연다
2. b01을 복사해 잠재_크기를 2로 바꾸고 실행한다. 시험 손실은 b02 표의 어느 칸과 같아야 할까? 되살린 사진은 어떻게 달라질까?

```
모델 = 오토인코더(잠재_크기=2)       # 32 → 2
격자_저장(torch.cat([X_시험[:8], 복원]), "ae_recon_2.png")
```

21차시 · 직접 실행 · 15-generative-models 폴더에서 · b01 27초, 바꾼 복사본은 이 컴퓨터에서 7초, b03 4초

오토인코더를 학습할 때 손실을 계산하는 정답은 무엇일까요?

Q

가 사진의 옷 종류 번호 0~9

나 입력 사진 자기 자신

다 PCA로 줄였다 되살린 사진

라 잠재 벡터 32칸

21차시 정리

생성 모델은 경계가 아니라 데이터가 어떻게 생겼는지(분포)를 배워 새 샘플을 뽑는다

오토인코더: 인코더로 줄이고 디코더로 되살린다. 정답은 입력 자신, 병목이 중요한 것만 남긴다

아주 작게 줄일 땐 PCA보다 낫다. **잠재 공간**에서는 정답 없이도 비슷한 것끼리 모인다

다음 차시: 합성곱 오토인코더로 잡음 지우기, 이상 탐지, 그리고 "그릴 수 있을까"

22차시

22

잡음 지우기, 이상 찾기, 그리기 시도

책 15.3 ~ 15.5절 · 시작하자마자 b05_denoise.py 실행 (73초)

학습 목표

- stride=2 합성곱으로 줄이고 전치 합성곱으로 키우는 모양을 계산한다
- 잡음 섞은 입력과 깨끗한 정답으로 잡음 제거기를 학습한다
- 재구성 오차와 문턱으로 처음 보는 것을 찾아낸다
- 오토인코더의 디코더로는 왜 새 그림이 잘 안 나오는지 숫자로 설명한다

15.3 줄이는 합성곱, 키우는 전치 합성곱

```
x = torch.zeros(1, 1, 28, 28)
줄이기 = nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1)
키우기 = nn.ConvTranspose2d(16, 1, kernel_size=4, stride=2, padding=1)
작아진 = 줄이기(x)
# 출력 크기 공식: (입력 - 1) × stride - 2 × padding + kernel_size
print("공식으로 계산:", (14 - 1) * 2 - 2 * 1 + 4)
```

```
Conv2d(stride=2)          : (1, 1, 28, 28) → (1, 16, 14, 14)
ConvTranspose2d(stride=2): (1, 16, 14, 14) → (1, 1, 28, 28)
공식으로 계산: 28
```

합성곱: 여러 칸을 보고 **한 칸**을 만든다

전치 합성곱: 한 칸을 **도장처럼** 여러 칸에 찍는다. 커널 4, 패딩 1, 보폭 2 = 정확히 두 배

15.3 잡음 제거 오토인코더

```
self.인코더 = nn.Sequential(
    nn.Conv2d(1, 16, 3, stride=2, padding=1), nn.ReLU(),      # (16, 14, 14)
    nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU())    # (32, 7, 7)
self.디코더 = nn.Sequential(
    nn.ConvTranspose2d(32, 16, 4, stride=2, padding=1), nn.ReLU(),  # (16, 14, 14)
    nn.ConvTranspose2d(16, 1, 4, stride=2, padding=1), nn.Sigmoid()) # (1, 28, 28)

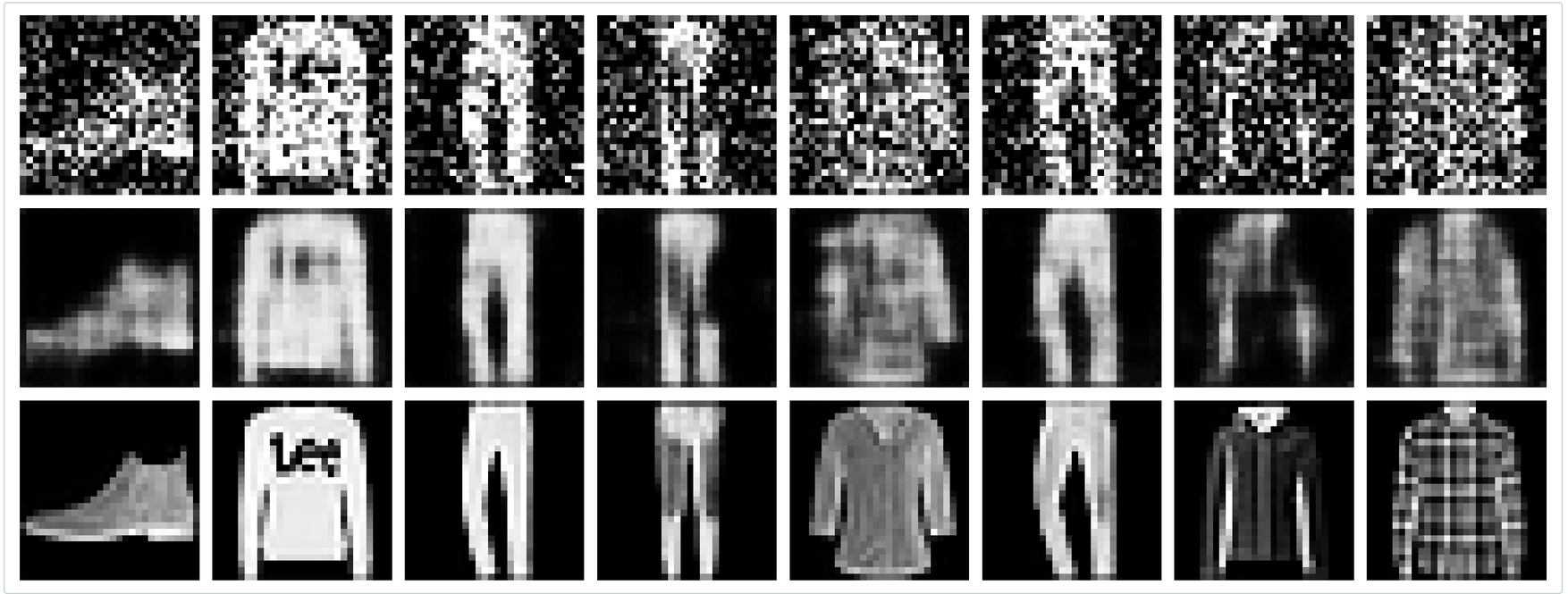
오토인코더_학습(모델, X_학습, 에폭=3, 잡음=0.4)   # 잡음 섞인 입력 → 깨끗한 원본

# 생성도구.py 안에서 배치마다
입력 = (x + 잡음 * torch.randn_like(x)).clamp(0, 1)
손실 = F.mse_loss(모델(입력), x)          # 정답은 언제나 깨끗한 원본
```

입력엔 **잡음 섞은 사진**, 정답엔 **깨끗한 원본**

"잡음을 걷어 내는 법을 배운다" = 확산 모델의 씨앗

15.3 잡음에 묻힌 옷을 되찾다



원본과의 차이(MSE) **0.0849 → 0.0161**, 5분의 1 남짓

파라미터 **13,265개**: 완전 연결(419,120개)의 30분의 1

15.4 재구성 오차로 이상한 것 찾기

정상 제품만 보고 배운 오토인코더는 **처음 보는 것**을 잘 못 되살린다

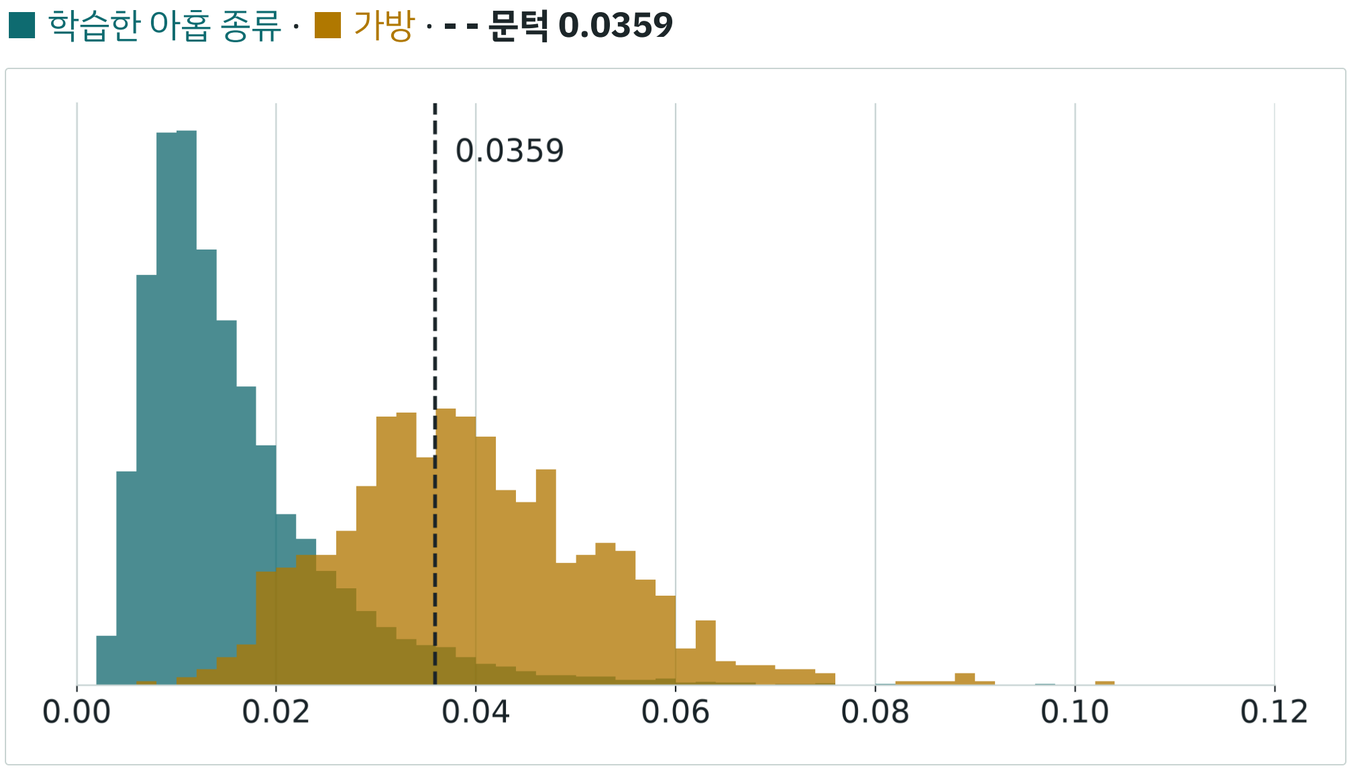
실험: **가방**을 불량품 역할로. 가방은 한 장도 안 보여 주고 아홉 종류로만 학습

```
모델 = 오토인코더_학습(오토인코더(잠재_크기=8), X_학습[y_학습 != 8],
                      에폭=5, 출력=False).eval()
with torch.no_grad():
    오차 = ((모델(X_시험) - X_시험) ** 2).flatten(1).mean(dim=1)  # 한 장마다

# 문턱: 학습한 종류의 시험 사진 중 95%가 넘지 않는 오차
문턱 = 오차[y_시험 != 8].quantile(0.95)
```

15.4 처음 보는 가방이 가장 못 되살아난다

종류	평균 오차
T셔츠	0.0159
바지	0.0097
풀오버	0.0163
드레스	0.0157
코트	0.0152
샌들	0.0247
셔츠	0.0165
운동화	0.0125
가방	**0.0395**
앵클부츠	0.0169



가로는 사진 한 장의 재구성 오차, 세로는 비율(두 묶음 각각 넓이 1)

22차시 · 15.4 재구성 오차로 이상한 것 찾기 · 표는 b06_anomaly.py 출력, 그림은 같은 설정으로 다시 계산한 오차 분포

15.4 문턱 하나로 자르면

문턱

0.0359

학습한 종류의 95%가 넘지 않는 값

잡아낸 가방

58.6%

네모난 가방은 셔츠·풀오버처럼 그럭저럭 되살아난다

괜히 의심한 옷

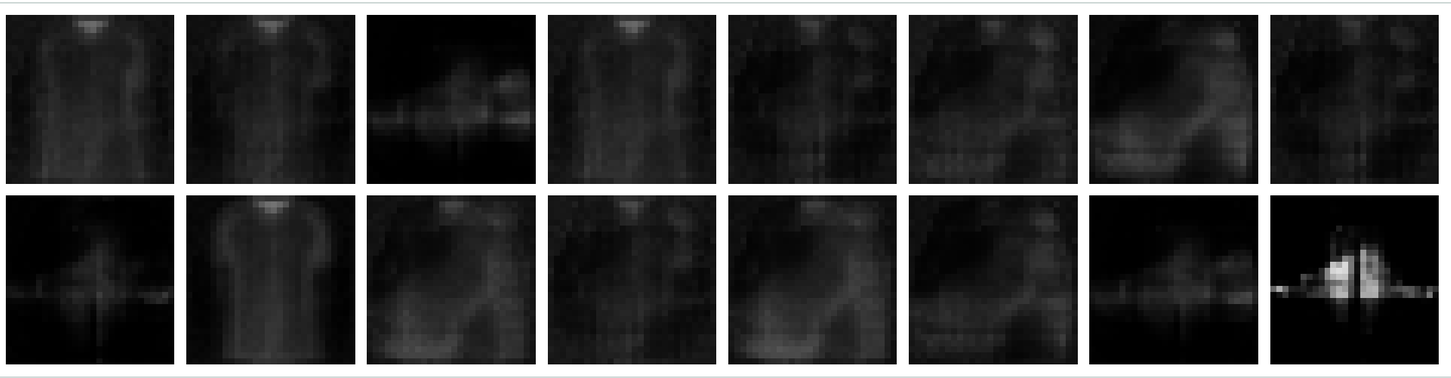
5.0%

문턱을 정할 때 이미 정해 둔 값

문턱을 올리고 내리며 **놓치는 불량**과 **괜한 의심** 사이에서 고른다: 정밀도와 재현율의 줄다리기

15.5 디코더에 아무 점이나 넣으면 새 옷이 나올까

```
아무_점 = torch.randn(15, 2)                # 평균 0, 표준편차 1에서 15개
빈_땅 = torch.tensor([[0.0, 7.0]])          # 마지막 칸: 데이터가 없는 곳
격자_저장(모델.디코더(torch.cat([아무_점, 빈_땅])), "ae_random.png")
```



```
잠재 공간 범위: 1번 칸 [-15.8, 17.1] / 2번 칸 [-11.3, 7.7]
잠재 벡터의 평균: [2.7, -2.37] 표준편차: [7.39, 3.53]
(0, 7)에서 거리 1 안에 있는 학습 사진: 0 장
```

22차시 · 15.5 오토인코더로 새 그림을 그릴 수 있을까 · 예제 b07_ae_random.py (핵심 부분, 출력, ae_random.png)

15.5 그리려면 잠재 공간이 이래야 한다

지금: 범위가 제멋대로

데이터는 평균 (2.7, -2.37), 표준편차 (7.39, 3.53)에 사는데 우리는 평균 0, 표준편차 1에서 뽑았다

필요 1: 정해진 동네

데이터가 평균 0, 표준편차 1 근처에 살아야 어디서 뽑을지 안다

지금: 빈 땅이 많다

(0, 7) 근처 학습 사진 0장. 디코더가 한 번도 받아 본 적 없는 점

필요 2: 빈 땅이 없다

동네 안 어느 점을 뽑아도 그럴듯한 그림이 나와야 한다

두 조건을 손실 함수로 강제한 것이 다음 차시의 VAE

직접 실행 10분

1. b05가 만든 denoise.png를 열고, b06_anomaly.py와 b07_ae_random.py를 실행한다
2. b06을 복사해 불량품 역할을 가방(8번) 대신 **바지(1번)**로 바꾼다. 바지는 가방보다 더 잘 잡힐까, 덜 잡힐까? 예상하고 실행한다

```
X_학습[y_학습 != 1]            # 8 → 1, 네 줄을 모두 바꾼다
문턱 = 오차[y_시험 != 1].quantile(0.95)
(오차[y_시험 == 1] > 문턱).float().mean()
```

ConvTranspose2d(kernel_size=4, stride=2, padding=1)에 7×7을 넣으면?

Q

가 4×4

나 7×7

다 14×14

라 28×28

22차시 정리

합성곱 오토인코더: stride=2로 줄이고 ConvTranspose2d로 키운다. 잡음 섞인 입력 → 깨끗한 정답이면 잡음 제거기

이상 탐지: 정상만 보고 배우면 낯선 것을 못 되살린다. 재구성 오차와 문턱으로 판정

오토인코더의 잠재 공간은 범위가 제멋대로이고 **빈 땅**이 많아 아무 점에서나 그리지 못한다

다음 차시: VAE, 점 대신 구름으로 · 그리고 GAN의 생각

23차시

23

VAE: 점 대신 구름으로

책 15.6 ~ 15.7절

학습 목표

- VAE의 손실이 재구성과 KL 발산 두 항인 이유를 말한다
- 재매개변수화로 무작위 뽑기를 미분하고, KL 발산을 직접 계산한다
- VAE로 새 그림을 그리고 잠재 공간 지도와 보간을 읽는다
- GAN의 생성자·판별자 경쟁과 약점을 설명한다

15.6 정확한 주소 대신 동네

오토인코더

사진마다 **점 하나**. 디코더는 그 주소만 외우면 되니 사이는 빈 땅

VAE

사진마다 **구름**(평균, 분산). 학습마다 그 안에서 점을 무작위로 뽑는다. 구름이 겹쳐 빈 땅이 메워진다

손실 항	하는 일	혼자 두면
재구성 손실	원본을 잘 되살리게 한다	구름을 점으로 줄이고 흩어 놓는다
KL 발산	구름을 원점 근처로 모으고 넓게 편다	모든 구름이 같아져 무엇을 넣든 같은 그림

15.6 무작위로 뽑으면서 미분하기: 재매개변수화

```
평균 = torch.tensor([0.5, -1.0], requires_grad=True)
로그분산 = torch.tensor([0.0, -2.0], requires_grad=True)
표준편차 = torch.exp(0.5 * 로그분산)
분포 = Normal(평균, 표준편차)
z = 분포.sample()                    # 1) 그냥 뽑기: 계산 기록이 없는 '상수'
잡음 = torch.randn(2)
z = 평균 + 표준편차 * 잡음           # 2) 무작위는 잡음에, z는 '계산'
z.sum().backward()
```

```
sample()  : tensor([ 1.1614, -0.9018]) | grad_fn: None
직접 계산 : tensor([ 2.0410, -1.1079]) | grad_fn: AddBackward0
rsample() : tensor([ 2.0410, -1.1079])
평균의 기울기: tensor([1., 1.])
로그분산의 기울기: tensor([ 0.7705, -0.0540])
```

z = 평균 + 표준편차 × 잡음: 같은 분포인데 기울기가 평균과 로그분산까지 흐른다

23차시 · 15.6 재매개변수화 · 예제 15-generative-models/b08_reparam.py (핵심 부분, 출력 일부)

15.6 KL 발산: 구름이 표준정규분포에서 얼마나 멀까

```
kl_공식 = -0.5 * (1 + 로그분산 - 평균 ** 2 - 로그분산.exp())
kl_divergence(분포, Normal(0.0, 1.0))
```

```
KL (공식)                : tensor([0.1250, 1.0677])
KL (torch.distributions): tensor([0.1250, 1.0677])
```

첫째 칸 · 평균 0.5, 분산 1

평균만 0.5 벗어남 → 0.125

둘째 칸 · 평균 -1, 분산 0.135

벗어나고 쪼그라듦 → 1.068

평균 0, 분산 1이면

벌점은 정확히 0

23차시 · 15.6 재매개변수화 · 예제 b08_reparam.py (마지막 부분과 출력)

15.6 VAE: 오토인코더에서 바뀐 세 군데

```
self.몸통 = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU())
self.평균 = nn.Linear(256, 잠재_크기)                # (1) 인코더가 둘로 갈린다
self.로그분산 = nn.Linear(256, 잠재_크기)

def forward(self, x):
    h = self.몸통(x)
    평균, 로그분산 = self.평균(h), self.로그분산(h)
    z = 평균 + torch.exp(0.5 * 로그분산) * torch.randn_like(평균)  # (2)
    return self.디코더(z), 평균, 로그분산

def vae_손실(로짓, x, 평균, 로그분산):                 # (3) 손실이 두 항
    재구성 = F.binary_cross_entropy_with_logits(로짓, x, reduction="sum") / len(x)
    kl = -0.5 * (1 + 로그분산 - 평균 ** 2 - 로그분산.exp()).sum() / len(x)
    return 재구성, kl
```

두 항 모두 칸별로 **더한 뒤** 배치 크기로 나눠 "사진 한 장당" 값으로 맞춘다. 한쪽만 평균을 내면 줄다리기의 균형이 무너진다

15.6 학습: 재구성은 줄고 KL은 자리를 잡는다

에폭	재구성	KL
1	304.1	7.86
2	272.3	6.58
5	263.7	6.41
10	**259.2**	**6.46**

값은 사진 한 장당 합계라 21차시의 0.0134와 눈금이 다르다

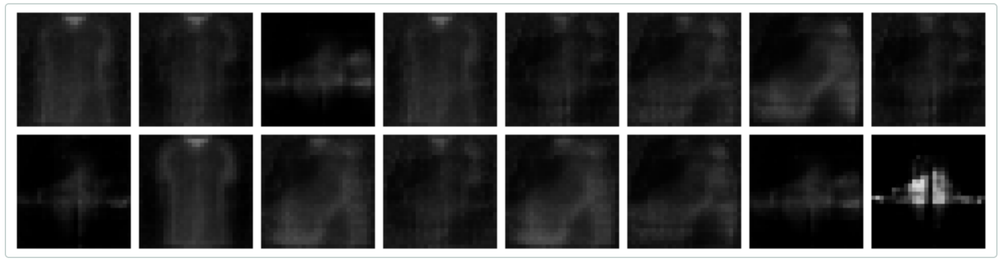
KL이 0이 아니다: 구름들이 표준정규분포와 완전히 같아지면 사진끼리 구별이 안 된다

```
# 표준정규분포에서 점을 뽑아 디코더에 넣으면 새 사진이 나온다
새_사진 = torch.sigmoid(모델.디코더(torch.randn(1000, 2)))
torch.save(새_사진, "vae_1000.pt")        # 24차시에 채점할 1,000장
```

23차시 · 15.6 VAE 학습하고 새 그림 그리기 · 예제 b09_vae_train.py 출력과 마지막 부분 (CPU 8초)

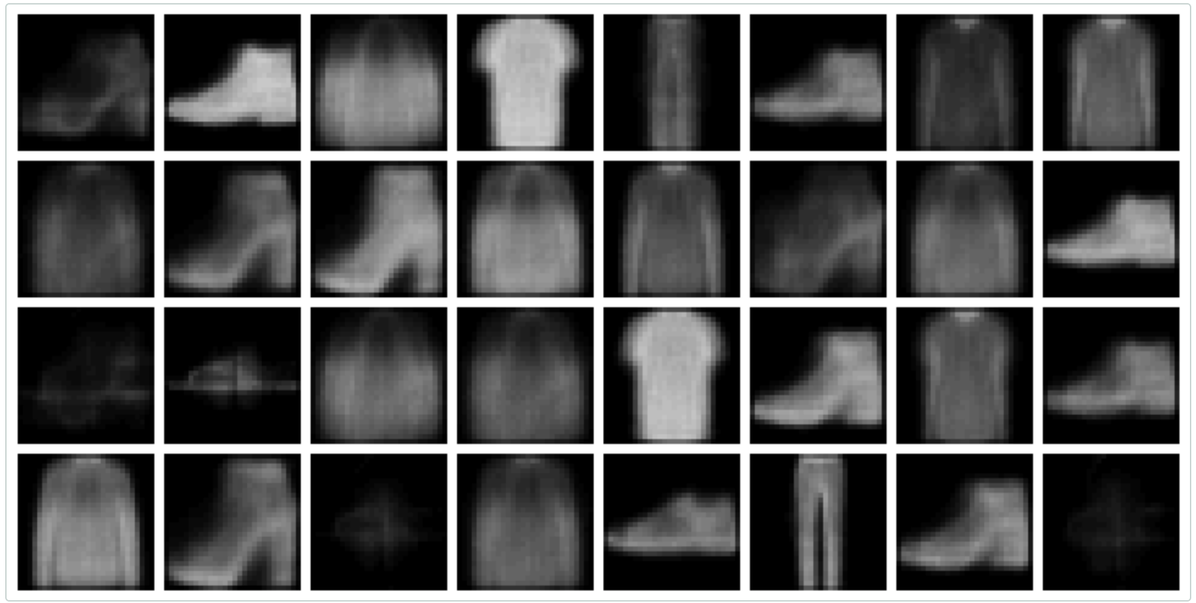
15.6 똑같이 torch.randn에서 뽑았는데

오토인코더 디코더 (22차시) · 16장



대부분 어두운 얼룩

VAE 디코더 · 32장

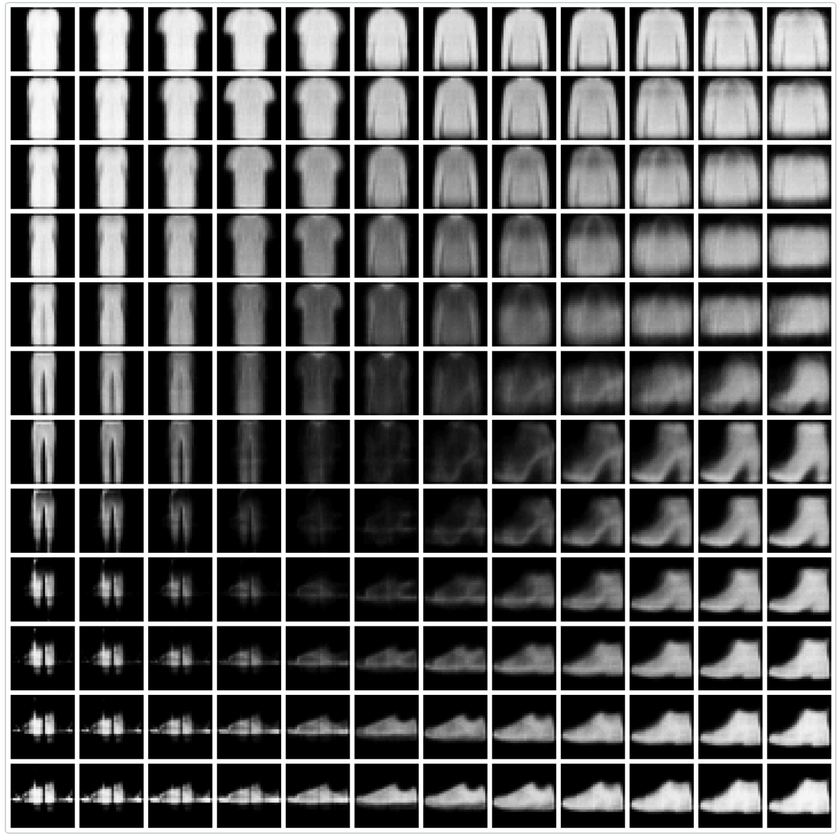


처음으로 **새 그림**: 앵클부츠, 운동화, T셔츠, 바지, 긴소매 옷

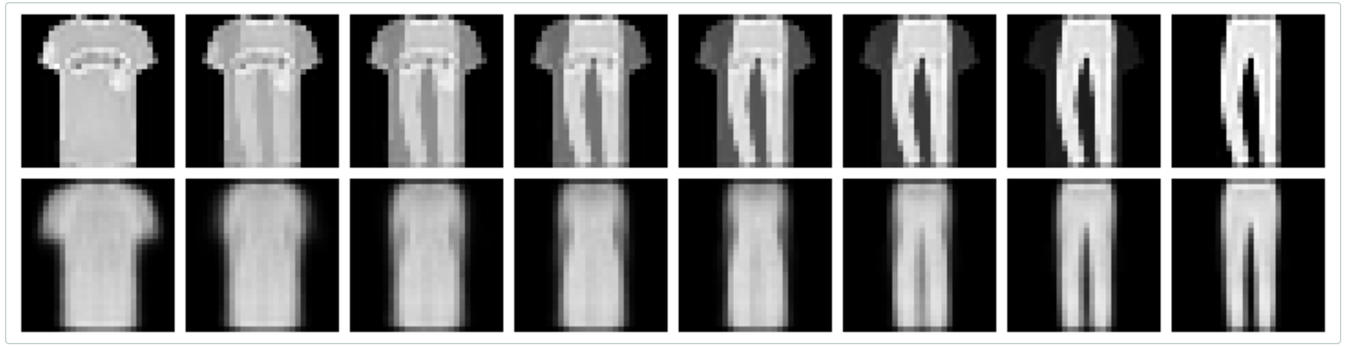
그러나 모두 **흐릿하다**: VAE의 대표적인 약점

23차시 · 15.6 VAE 학습하고 새 그림 그리기 · b07이 만든 ae_random.png와 b09가 만든 vae_samples.png

15.6 잠재 공간 지도와 보간: 빈 땅이 없다



위: 픽셀을 섞기 · 아래: 잠재 공간에서 옮겨 가기 (T셔츠 → 바지)



지도 어느 칸에서도 무언가 옷이 나오고 이웃 칸끼리는 조금씩만 다르다

(-1.15, 1.8) → (-2.86, -0.01): 중간 그림이 **그 자체로 그럴듯한 옷**

23차시 · 15.6 잠재 공간 지도와 보간 · 예제 b10_vae_map.py가 만든 vae_map.png, interp.png와 출력

15.7 GAN: 위조범과 감별사

생성자 = 위조범

잡음을 받아 그림을 그린다. VAE 디코더 자리. 목표: 감별사 속이기

판별자 = 감별사

그림 한 장이 진짜인지 가짜인지 맞힌다. 6장의 이진 분류기 모양

잘 굴러가면 **선명한** 그림: 흐릿한 그림은 감별사가 바로 알아본다

굴러가기가 어렵다: 한쪽이 너무 강하면 멈추고, **모드 붕괴** (운동화만 그리기)

15.7 세 생성 모델 나란히

	VAE	GAN	확산 모델
학습 방식	되살리기 + KL 벌점	생성자와 판별자의 경쟁	잡음 맞히기 (MSE 하나)
학습 안정성	안정적	까다로움	안정적
그림 품질	흐릿한 편	선명함	선명함
다양성	좋음	모드 붕괴 위험	좋음
그리는 속도	디코더 한 번	생성자 한 번	수십~수천 단계 (느림)

확산 모델이 GAN의 자리를 넘겨받은 이유: 학습은 **MSE 하나로 안정적**인데 그림은 **선명하고 다양하다**. 대가는 느린 그리기

직접 실행 10분

1. b08, b09, b10을 차례로 실행하고 vae_samples.png, vae_map.png, interp.png를 연다
2. b09와 b10을 복사해 KL에 4를 곱한 판과 0.25를 곱한 판을 만든다(베타-VAE). 마지막 에폭의 재구성과 KL, 지도는 어떻게 달라질까?

```
(재구성 + 4 * kl).backward()        # 또는 0.25 * kl
torch.save(모델.state_dict(), "vae_b4.pt")  # vae.pt는 24차시에 쓰니 덮어쓰지 않는다
```

23차시 · 직접 실행 · 학습 복사본은 이 컴퓨터에서 11~17초, 지도 복사본 1~2초 · 저장 파일 이름을 모두 바꾼다

재매개변수화를 쓰는 가장 큰 이유는?

Q

가 뽑기를 더 빠르게 하려고

나 뽑은 z를 거쳐 평균과 분산까지 기울기가 흐르게 하려고

다 분산이 음수가 되지 않게 하려고

라 KL 발산을 0으로 만들려고

23차시 정리

VAE: 점 대신 평균·로그분산의 구름. 손실 = 재구성 + KL 발산의 줄다리기

재매개변수화 z = 평균 + 표준편차 × 잡음으로 뽑기를 미분한다

아무 점에서나 그리고 보간이 매끄럽지만 흐리다. GAN은 선명하지만 학습이 까다롭다

다음 차시: 확산 모델을 처음부터 만들어 그리고, 그린 그림을 채점한다

24차시

24

확산 모델: 잡음을 걷어 내며 그리기

책 15.8 ~ 15.9절 · 시작하자마자 b13 학습(110초), 이어서 b14 그리기(45초)

학습 목표

- 잡음을 한 번에 섞는 공식과 "섞인 잡음 맞히기" 학습을 설명한다
- 시간 임베딩과 작은 U-Net으로 된 잡음 예측기를 읽는다
- 1,000단계를 거꾸로 걸어 그림을 그리고 중간 과정을 읽는다
- 품질·다양성·새로움으로 생성 모델을 점검한다

15.8 잉크 한 방울이 퍼지는 과정을 거꾸로

앞으로: 잡음 더하기

사진에 잡음을 조금씩 1,000번 섞어 순수한 잡음으로.
학습이 필요 없는 정해진 계산

거꾸로: 잡음 빼기

순수한 잡음에서 출발해 조금씩 걷어 내며 사진으로. 이 "조금 걷어 내기"를 **신경망이 배운다**

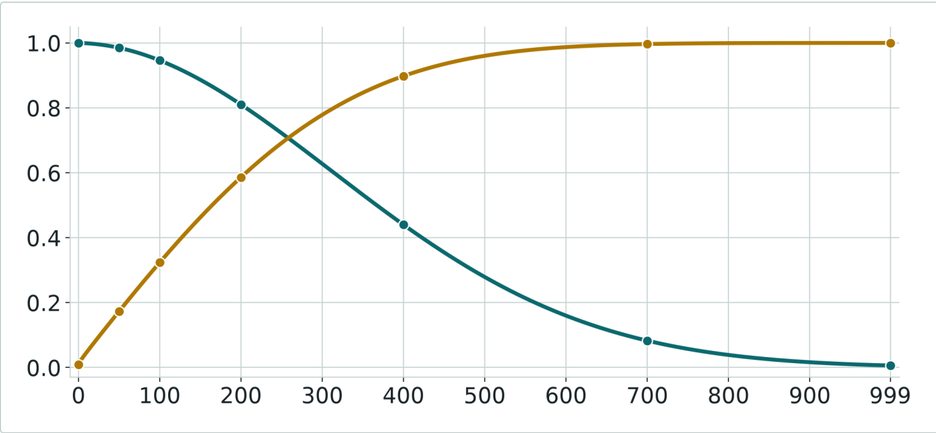
신경망에게 내는 문제

"이 사진에 섞인 **잡음**이 무엇이었는지 맞혀라" · 섞인 사진과 단계 t를 주고, 손실은 MSE 하나

15.8 앞으로 가는 과정: 잡음을 한 번에 섞는 공식

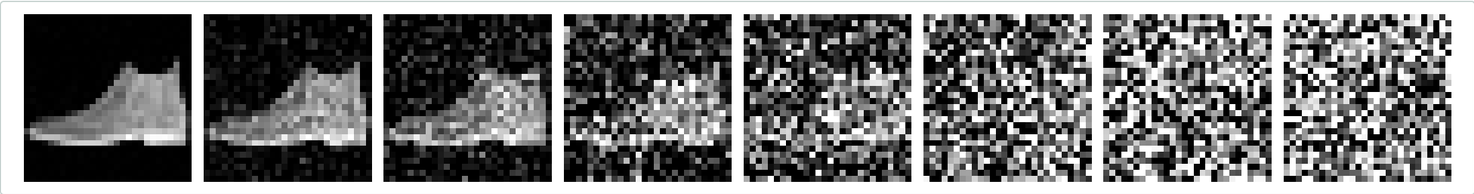
섞인 사진 = $\sqrt{a}$ × 원본 + $\sqrt{(1-a)}$ × 잡음 (a = 알파_누적[t], 알파를 0~t단계까지 곱한 값)

● 원본 비율 √a · ● 잡음 비율 √(1-a) · 가로는 단계 t



베타: 0.0001에서 0.02까지 1,000개, 알파 = 1 - 베타

한 걸음씩 300번 섞은 표준편차 0.7766, 공식 0.7769



24차시 · 15.8 앞으로 가는 과정 · 예제 b11_forward.py의 표를 곡선으로 다시 계산한 그림 · 아래는 forward.png (0~999단계)

15.8 확산도구.py: 배운 부품의 조합

부품	하는 일	어디서 본 것
잡음 일정	베타 1,000개, 알파_누적 = cumprod(알파)	4장 linspace
잡음_더하기	앞 장의 공식 한 줄, 사진마다 다른 t	4.6절 브로드캐스팅
시간_임베딩	단계 t를 여러 빠르기의 sin·cos 128칸으로	12장 위치 인코딩
블록	합성곱 두 번 + 시간 더하기 + 지름길	9장 ResNet 잔차 연결
작은UNet	28 → 14 → 7 → 14 → 28, 같은 크기끼리 이어 붙이기	15.3 전치 합성곱
그리기	잡음에서 출발해 999 → 0단계로 거꾸로	5.8절 no_grad

```
모양: (5,) → (5, 128)
t= 10 와 t= 11: 0.970
t= 10 와 t=500: 0.285
t= 10 와 t=999: 0.167
```

시간 임베딩의 코사인 유사도: 가까운 단계는 비슷하고, 먼 단계는 다르다

24차시 · 15.8 확산도구.py: 잡음 일정, 잡음 예측기, 그리기 · 예제 확산도구.py, b12_time_embed.py (출력 일부)

15.8 학습시키기: 섞은 잡음을 맞혀라

```
X = (X * 2 - 1).to(장치)                    # -1~1로, 통째로 장치에 (약 188MB)
for 걸음 in range(1, 5001):
    x0 = X[torch.randint(0, len(X), (128,), device=장치)]  # 사진 128장 무작위로
    t = torch.randint(0, T, (128,), device=장치)          # 사진마다 단계도 무작위로
    잡음 = torch.randn_like(x0)
    손실 = F.mse_loss(모델(잡음_더하기(x0, t, 잡음), t), 잡음)
    옵티마이저.zero_grad(); 손실.backward(); 옵티마이저.step()
    스케줄러.step()             # CosineAnnealingLR(T_max=5000)
```

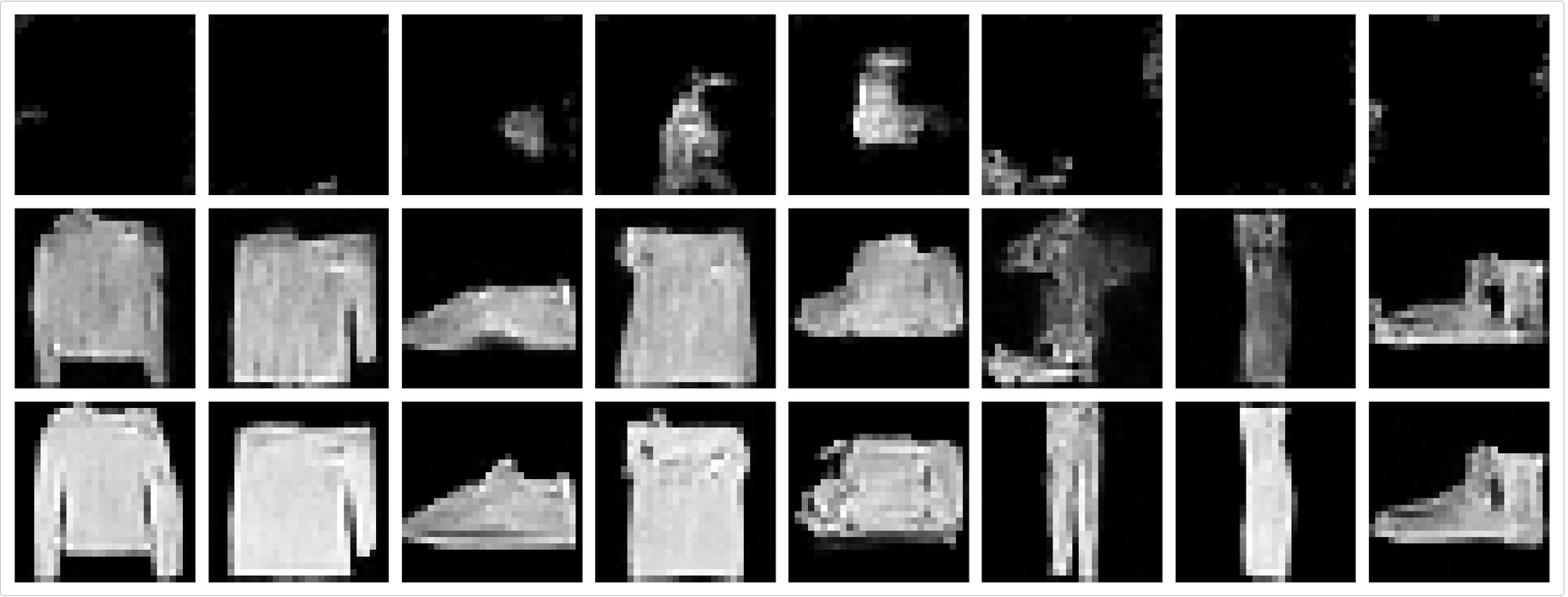
```
장치 mps, 파라미터 546,753개
 1000걸음: 최근 1000걸음 평균 손실 0.0641
 3000걸음: 최근 1000걸음 평균 손실 0.0423
 5000걸음: 최근 1000걸음 평균 손실 0.0388
```

"모른다, 0"이라고 답하면 MSE 1. 그 25분의 1 아래

마지막 가중치로 그리니 학습률을 0 가까이 줄여 끝낸다

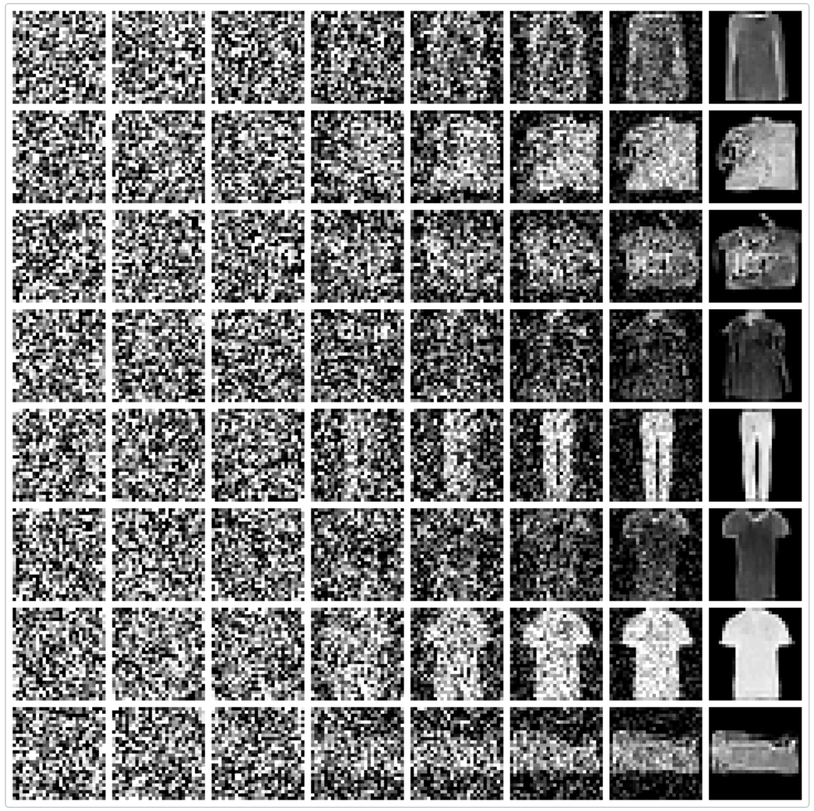
24차시 · 15.8 학습시키기 · 예제 b13_diffusion_train.py (핵심 부분, 출력 일부) · 이 컴퓨터 GPU(MPS)로 110초

15.8 학습 중간에 그려 보기: 손실보다 그림이 말한다



같은 씨앗으로 그려서 **같은 칸 = 같은 잡음에서 출발**. 생성 모델은 학습 중간에 그려서 눈으로 확인하는 것이 기본 습관

15.8 그리기: 잡음에서 옷이 나오기까지



한 줄 = 한 장 · 왼쪽부터 t = 999, 800, 600, 400, 300, 200, 100, 0

단계 t	값 범위	표준편차
999	-3.67 ~ 3.56	1.01
600	-3.28 ~ 3.44	0.99
400	-3.65 ~ 3.43	0.93
300	-3.29 ~ 2.88	0.87
200	-3.11 ~ 2.36	0.78
100	-2.14 ~ 1.64	0.67
0	-1.05 ~ 1.31	0.62

999~400 거의 잡음 · **300~200 윤곽** · 100부터 세부 다듬기

24차시 · 15.8 그리기 · 예제 b14_diffusion_sample.py가 만든 diffusion_reverse.png와 출력(800단계 줄 생략) · GPU 45초

15.8 새 그림 64장: VAE보다 훨씬 또렷하다



가방 손잡이, 샌들 끈, 상의 소매 같은 **세부**가 보인다

완벽하진 않다: 얼룩덜룩한 무늬, 상의인지 가방인지 모호한 그림, 무너진 샌들

키울 손잡이	이 장의 값	키우면
학습 걸음 수	5,000 (약 11에폭)	가장 확실하게 좋아짐
채널	32	세부 무늬가 좋아짐
가중치 이동 평균	없음	그림이 깨끗해짐
그리는 단계 수	1,000	줄이면 빠르다(DDIM 50)

24차시 · 15.8 그리기, 무엇을 키우면 좋아질까 · 예제 b14_diffusion_sample.py가 만든 diffusion_samples.png · 표는 책의 표 일부

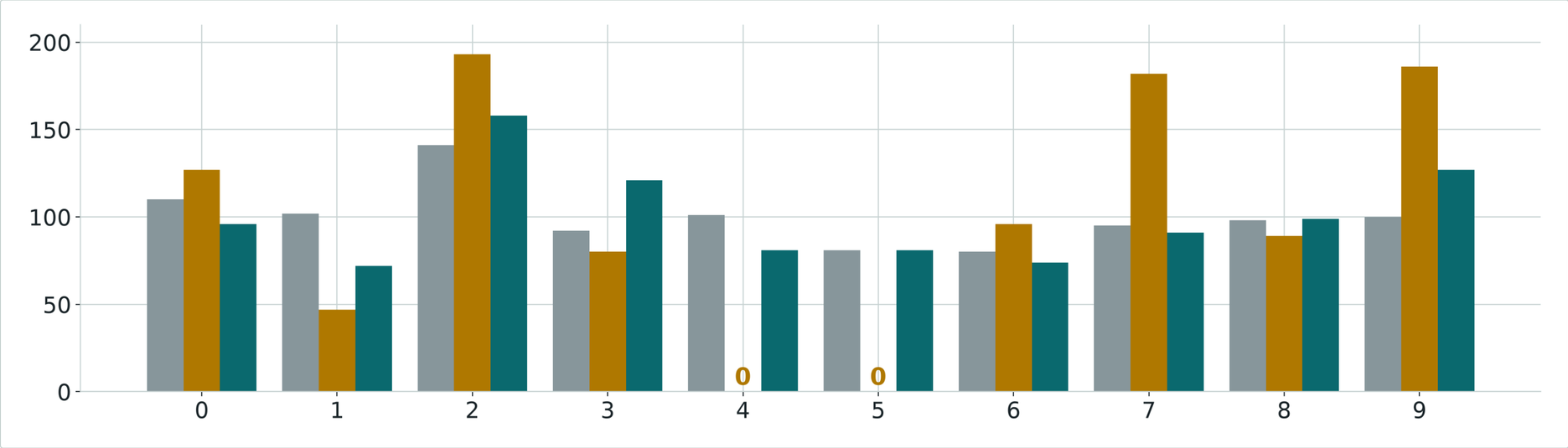
15.9 정답이 없는 시험: 채점기로 따라 하는 FID의 생각

확신도(품질) · 진짜 사진
88.1%

확신도 · 확산 모델
80.3%

확신도 · VAE
61.2%

종류별 개수(다양성), 1,000장씩: 진짜 사진 · VAE · 확산 모델

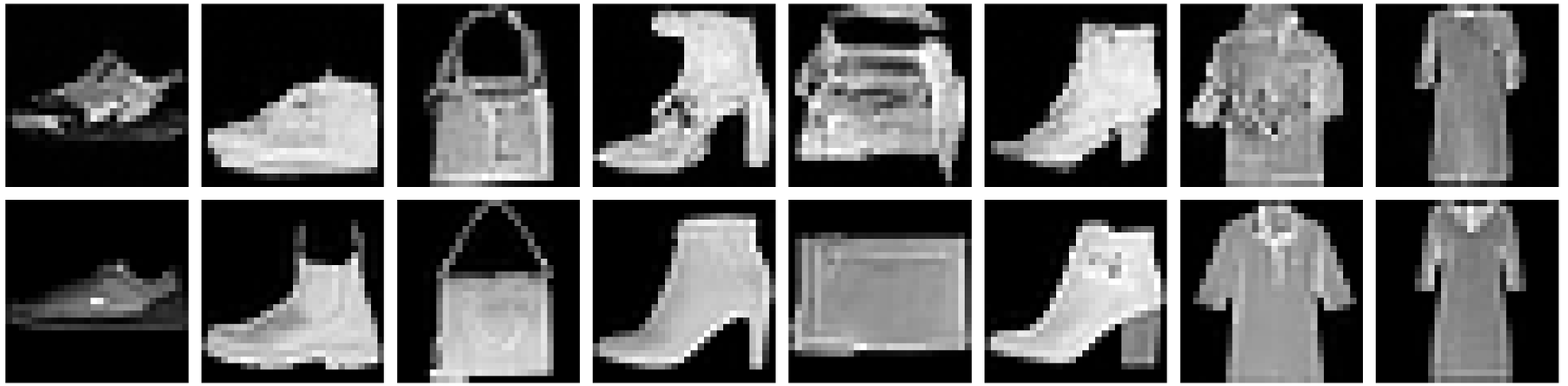


0 T셔츠 1 바지 2 풀오버 3 드레스 4 코트 5 샌들 6 셔츠 7 운동화 8 가방 9 앵클부츠 · VAE는 코트와 샌들이 0장

15.9 베낀 건 아닐까: 가장 가까운 학습 사진

그린 그림

가장 가까운 학습 사진



평균 거리: 진짜 시험 사진 3.43, 확산 모델 4.25. 베꼈다면 0에 가깝다

마지막 심사위원은 **사람의 눈**: 채점기도 틀릴 수 있는 간이 점검

직접 실행 10분

1. b11, b12를 실행하고, b14가 끝났으면 b15_evaluate.py로 채점 숫자를 확인한다
2. b13을 복사해 걸음을 **1,000**으로 줄인 실습판을 만든다. 1,000걸음 모델의 그림은 책 그림의 몇 번째 줄과 비슷할까?

```
스케줄러 = ...CosineAnnealingLR(옵티마이저, T_max=1000)  # 5000 → 1000
for 걸음 in range(1, 1001):                          # 함께 바꾼다
    if 걸음 in (200, 500, 1000):           # 중간 그림 시점
torch.save(모델.state_dict(), "확산_1000걸음.pt")  # 확산.pt를 덮어쓰지 않게
```

24차시 · 직접 실행 · 실습판은 GPU(MPS) 30초, CPU만 쓰면 29분이라 GPU 없는 자리는 강사 시연 · b11 9초, b12 1초 미만, b15 4초

확산 모델의 신경망이 학습 중에 맞히는 것은?

Q

가 사진의 옷 종류

나 섞인 사진에 들어간 잡음

다 진짜 사진인지 가짜인지

라 섞은 단계 t

24차시 정리

확산 모델: 잡음을 한 번에 섞는 공식으로 학습 데이터를 만들고, 섞인 잡음을 MSE로 맞힌다

시간 임베딩 + **U-Net**(이어 붙이기, 잔차 연결, GroupNorm). 그릴 때는 1,000단계를 거꾸로 걷는다

평가는 품질·다양성·새로움으로 쪼갠다. 마지막 심사위원은 사람의 눈

다음 차시: "바지를 그려 줘" 주문하는 조건부 확산 실습과 과정 마무리

25차시

25

실습: 주문한 옷을 그리는 조건부 확산

책 15.10 ~ 15.11절 · 시작하자마자 b16 학습(123초), 이어서 b17 채점(47초)

학습 목표

- 종류 임베딩을 시간 임베딩에 더해 "무엇을 그릴지" 알려 준다
- 채점기로 주문을 얼마나 지켰는지 종류별로 재고 읽는다
- 5일 과정에서 다시 만난 부품들로 전체를 돌아본다

15.10 바꿀 곳은 두 군데

```
# 확산도구.py의 작은UNet 안에 이미 있던 손잡이
self.클래스_층 = nn.Embedding(클래스_수, 시간_크기) if 클래스_수 > 0 else None
시간 = 시간 + self.클래스_층(클래스)          # 무엇을 그릴지도 함께 알려 준다

모델 = 작은UNet(클래스_수=10).to(장치)     # 바뀐 곳 (1)
손실 = F.mse_loss(모델(잡음_더하기(x0, t, 잡음), t, 종류), 잡음)  # 바뀐 곳 (2)

종류 = torch.arange(10, device=장치).repeat_interleave(8)  # [0, 0, …, 9]
그림, _ = 그리기(모델, 80, 장치, 클래스=종류)
```

```
1000걸음: 평균 손실 0.0622
3000걸음: 평균 손실 0.0427
5000걸음: 평균 손실 0.0406
```

주문서가 "종류 번호" 대신 **"문장"**이면 그림 생성 AI

15.10 주문대로 나왔을까: 한 줄이 한 종류



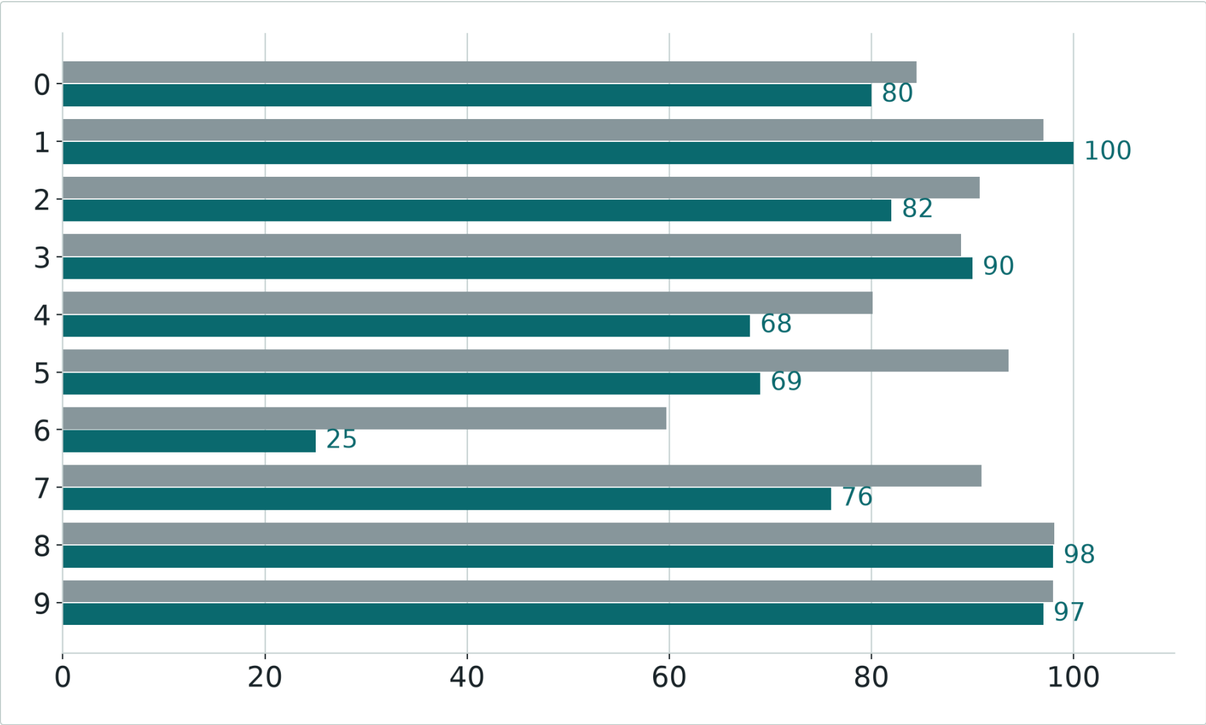
바지, 운동화, 가방, 앵클부츠 줄: 거의 모두 주문대로

T셔츠 줄엔 소매 짧은 상의, 풀오버·코트 줄엔 긴소매 상의

셔츠 줄은 T셔츠 같기도 풀오버 같기도. 샌들 줄은 끈이 어지럽게 엉켰다

15.10 채점기로 확인: 주문대로 그린 비율

■ 주문대로 그린 비율(%) · ■ 채점기의 진짜 사진 정확도



0 T셔츠 1 바지 2 풀오버 3 드레스 4 코트 5 샌들 6 셔츠 7 운동화 8 가방 9 앵클부츠

전체

78.5% / 88.2%

셔츠 25%: 채점기도 진짜 셔츠를 60%만 맞힌다

샌들 69% 대 94%: 가는 끈은 작은 모델이 아직 못 그린다

15.10 더 해 볼 만한 것

주문을 더 잘 지키게

10%는 종류를 "없음"(11번째 번호)으로. 그릴 때 두 예측의 차이를 키운다

분류기 없는 가이던스(CFG)

손잡이 키우기

걸음 수, 채널, EMA를 늘리고 같은 채점기로 점수가 얼마나 오르는지 잰다

실험 기록은 18장

MNIST로 바꾸기

FashionMNIST → MNIST면 손글씨 숫자를 주문해 그린다. 무늬가 없어 더 깔끔하다

데이터를 바꿀 땐 출처와 라이선스부터

직접 실행 10분

1. b16을 복사해 걸음을 **1,000**으로 줄이고(200걸음마다 출력), 저장 이름을 바꿔 실행한다
2. b17을 복사해 그 모델을 불러오고 종류마다 **20장**씩만 주문해 채점한다. 5,000걸음 모델의 78.5%와 비교하면?

```
for 걸음 in range(1, 1001):                  # 학습 복사본
torch.save(모델.state_dict(), "조건부_1000걸음.pt")
주문 = torch.arange(10, device=장치).repeat_interleave(20)  # 채점 복사본
그림, _ = 그리기(모델, 200, 장치, 클래스=주문)
```

25차시 · 직접 실행 · GPU(MPS)로 학습 복사본 31초, 채점 복사본 14초 · GPU 없는 자리는 강사 시연

셔츠를 주문대로 그린 비율이 25%뿐이었다. 가장 먼저 할 일은?

Q

가 확산 모델을 더 크게 키운다

나 채점기가 진짜 셔츠를 얼마나 맞히는지 본다

다 셔츠를 학습 데이터에서 뺀다

라 그리는 단계를 100으로 줄인다

15장 정리: 고르는 모델에서 그리는 모델로

모델	핵심	오늘 본 숫자
오토인코더	줄였다가 되살리기, 정답은 입력 자신	시험 손실 0.0134
잡음 제거·이상 탐지	잡음 → 깨끗한 정답 · 재구성 오차와 문턱	0.0849 → 0.0161 · 가방 58.6%
VAE	구름 + 재매개변수화 + KL, 아무 점에서나 그린다	확신도 61.2%
GAN	생성자와 판별자의 경쟁, 모드 붕괴 위험	(구현하지 않음)
확산 모델	섞인 잡음을 MSE로 맞히고 1,000단계를 거꾸로	확신도 80.3%
조건부 확산	종류 임베딩을 시간 임베딩에 더해 주문	주문대로 78.5%

평가는 **품질·다양성·새로움**으로 쪼개고, 측정 도구의 실력부터 확인한다

5일 동안 걸어온 길

1일차	**11장 RNN과 LSTM**	순서를 메모장에 고쳐 쓰며 읽고, 기억을 벨트로 따로 나른다
2일차	**12장 트랜스포머**	어텐션으로 모든 것을 서로 비교하고, 작은 GPT를 조립한다
3일차	**13장 한국어 처리**	어디서 끊을지 고르고(토큰화), 사전학습 인코더를 미세조정한다
4일차	**14장 언어 모델과 LoRA**	다음 토큰 예측 하나로 글을 쓰고, 고칠 부분만 작게 미세조정한다
5일차	**15장 생성 모델**	줄였다 되살리고, 구름으로 그리고, 잡음을 걷어 내며 그린다

같은 부품이 계속 돌아왔다

부품	처음 만난 곳	오늘 다시 만난 곳
nn.Embedding	11.11 글자를 벡터로 · 13장 단어를 벡터로	15.10 옷 종류를 벡터로
sin·cos 펼치기	12.6 단어의 위치 정보	15.8 잡음 단계 t의 시간 임베딩
잔차 연결	9장 ResNet · 12.7 트랜스포머 블록	15.8 U-Net의 블록
조건 붙여 주문하기	14.4 프롬프트와 채팅 템플릿	15.10 "바지를 그려 줘"
작게 줄여 담기	14.7 LoRA: 고칠 부분만 작게	15.1 병목: 중요한 것만 담기

새 모델은 대개 **아는 부품의 새 조합**이다: 처음 보는 논문도 부품부터 찾으면 읽힌다

과정 마무리 · 다시 만난 부품들 · 각 칸의 절 번호는 책 2권 기준

과정을 마치며

1. 순서는 **메모장(RNN)**으로, 관계는 **어텐션**으로 읽는다
2. 한국어는 **어디서 끊을지**부터 정하고 사전학습 모델을 빌린다
3. 큰 언어 모델은 **다음 토큰 예측** 하나, 고칠 땐 LoRA로 작게
4. 그리는 모델은 **분포**를 배우고, 확산은 잡음을 걷어 내며 그린다
5. 손실보다 **출력을 직접 보고**, 측정 도구의 실력부터 확인한다

다음 걸음 · 책 16장부터

- 16장 GPU와 성능: 혼합 정밀도, torch.compile, 프로파일링
- 17장 분산·대규모 학습
- 18장 실험 관리와 재현성
- 19~20장 ONNX·양자화, FastAPI 서빙